

CEIPA BUSINESS SCHOOL

Ingeniería de Sistemas

Actividad 3-1

Robustez con Encapsulamiento, Herencia y Polimorfismo (POO)

Fundamentos de Software

Estudiante:	Heyller De Jesús Galeano Guarín
Docente:	Simón Peláez Loaiza
Curso:	Fundamentos de Software
Actividad:	3-1
Fecha:	Julio de 2026

Resumen

Este informe expone cómo los tres pilares de la Programación Orientada a Objetos —Encapsulamiento, Herencia y Polimorfismo— se aplican de manera conjunta para incrementar la robustez de un sistema de software. Tomando como caso de estudio un sistema de gestión de cuentas y transacciones, se muestra cómo cada pilar previene estados inválidos, reduce la duplicación de código y permite extender el sistema sin comprometer su estabilidad. El resultado es una arquitectura más resiliente frente a errores de uso, datos inconsistentes y futuros cambios de requisitos.

Introducción

La robustez de un sistema se mide por su capacidad de operar correctamente incluso ante entradas inesperadas, condiciones límite o errores de los usuarios que lo consumen. La Programación Orientada a Objetos ofrece mecanismos concretos para lograr esta robustez: el encapsulamiento protege el estado interno de los objetos, la herencia centraliza reglas comunes evitando su duplicación (y por tanto su posible inconsistencia), y el polimorfismo permite que el sistema reaccione de forma adecuada ante distintos tipos de objetos sin necesidad de bifurcaciones condicionales frágiles. En esta actividad se analiza e implementa cada uno de estos pilares con foco explícito en la prevención de errores y el manejo controlado de excepciones.

Objetivo general

Diseñar e implementar un sistema orientado a objetos robusto, aplicando Encapsulamiento, Herencia y Polimorfismo como mecanismos de control, reutilización y extensibilidad frente a errores y cambios futuros.

Objetivos específicos

- Proteger el estado interno de los objetos mediante atributos privados y propiedades con validación.
- Centralizar reglas de negocio comunes en una clase base para evitar duplicación e inconsistencias.
- Aplicar polimorfismo para que distintos tipos de cuenta o transacción respondan de forma segura a una misma interfaz.
- Incorporar manejo de excepciones que impida que datos inválidos comprometan la integridad del sistema.

Marco conceptual

El Encapsulamiento consiste en ocultar los detalles internos de un objeto y exponer únicamente una interfaz controlada de acceso. La Herencia permite que una clase (subclase) reutilice atributos y comportamientos definidos en otra (superclase), evitando duplicar lógica común. El Polimorfismo permite que objetos de distintas clases respondan a un mismo mensaje o método de forma específica según su propio comportamiento, sin que el código cliente necesite conocer el tipo concreto del objeto.

Desarrollo

Encapsulamiento: protección del estado interno

Para evitar que el saldo de una cuenta quede en un estado inválido, los atributos críticos se declaran privados y se exponen únicamente mediante propiedades. El setter de saldo valida el valor antes de asignarlo y lanza una excepción controlada si la operación pondría la cuenta en un estado inconsistente, en lugar de permitir silenciosamente un dato corrupto.

```
class CuentaBancaria:
    def __init__(self, titular, saldo_inicial=0):
        self._titular = titular
        self._saldo = 0
        self.saldo = saldo_inicial # pasa por el setter validado

    @property
    def saldo(self):
        return self._saldo

    @saldo.setter
    def saldo(self, valor):
        if valor < 0:
            raise ValueError("El saldo no puede ser negativo")
        self._saldo = valor

    def retirar(self, monto):
        if monto <= 0:
            raise ValueError("El monto a retirar debe ser positivo")
        if monto > self._saldo:
            raise ValueError("Fondos insuficientes")
        self.saldo -= monto
```

Figura 1. Atributos privados y validación mediante propiedades.

Herencia: reglas comunes centralizadas

Las cuentas de ahorro y corriente comparten la lógica base de depósito, retiro y validación de titular. Esta lógica se concentra en una clase base CuentaBase, evitando que cada subclase reimplemente —y potencialmente desincronice— las mismas reglas de negocio.

```
class CuentaBase:
    def __init__(self, titular, saldo_inicial=0):
```

```

        if not titular:
            raise ValueError("La cuenta requiere un titular")
        self._titular = titular
        self._saldo = saldo_inicial

    def depositar(self, monto):
        if monto <= 0:
            raise ValueError("El depósito debe ser positivo")
        self._saldo += monto

class CuentaAhorros(CuentaBase):
    TASA_INTERES = 0.02

    def calcular_rendimiento(self):
        return self._saldo * self.TASA_INTERES

class CuentaCorriente(CuentaBase):
    SOBREGIRO_MAXIMO = 200000

    def retirar(self, monto):
        if monto > self._saldo + self.SOBREGIRO_MAXIMO:
            raise ValueError("Excede el sobregiro permitido")
        self._saldo -= monto

```

Figura 2. Clase base con reglas comunes y subclasses especializadas.

Polimorfismo: comportamiento seguro por tipo

Cada subclase redefine el método `procesar_cierre_mes()` según sus propias reglas, pero el código cliente invoca siempre la misma interfaz. Esto evita bloques condicionales frágiles del tipo `if tipo == "ahorros"` y hace que agregar un nuevo tipo de cuenta no requiera modificar el ciclo de procesamiento existente.

```

def procesar_cierre_mes(cuentas):
    for cuenta in cuentas:
        try:
            cuenta.procesar_cierre_mes()
        except ValueError as error:
            print(f"No se pudo cerrar la cuenta: {error}")

# CuentaAhorros.procesar_cierre_mes() aplica intereses.
# CuentaCorriente.procesar_cierre_mes() valida el sobregiro.
# Ambas comparten la misma firma heredada de CuentaBase.

```

Figura 3. Una misma interfaz, comportamientos distintos y seguros.

Manejo de errores y pruebas de robustez

Se realizaron pruebas deliberadas con datos inválidos —montos negativos, retiros superiores al saldo disponible y cuentas sin titular— para verificar que el sistema respondiera con excepciones controladas

(`ValueError`) en lugar de permitir estados inconsistentes o fallar de forma inesperada. En todos los casos el sistema rechazó la operación y mantuvo el estado previo del objeto intacto, confirmando la efectividad de la validación encapsulada.

Relación con principios SOLID

La combinación de los tres pilares refuerza distintos principios SOLID: el encapsulamiento reduce la fragilidad asociada al acceso directo a datos; la herencia bien aplicada favorece el principio de Sustitución de Liskov (LSP), ya que las subclases pueden usarse en cualquier lugar donde se espera la clase base sin romper el comportamiento; y el polimorfismo favorece el principio Abierto/Cerrado (OCP), permitiendo extender el sistema con nuevos tipos de cuenta sin modificar el código que ya funciona.

Encapsulamiento → Bajo acoplamiento	El acceso controlado a <code>_saldo</code> evita que módulos externos corrompan el estado de la cuenta.
Herencia → LSP	<code>CuentaAhorros</code> y <code>CuentaCorriente</code> pueden sustituir a <code>CuentaBase</code> sin alterar el comportamiento esperado del sistema.
Polimorfismo → OCP	Nuevos tipos de cuenta se agregan heredando de <code>CuentaBase</code> , sin modificar <code>procesar_cierre_mes()</code> .

Conclusiones

1. El encapsulamiento evita que datos inválidos ingresen al sistema, protegiendo su integridad desde el punto de entrada de cada operación.
2. La herencia centraliza las reglas comunes, evitando que la duplicación de lógica se convierta en una fuente de errores inconsistentes.
3. El polimorfismo permite extender el sistema con nuevos tipos de cuenta sin modificar el código existente, reduciendo el riesgo de regresiones.
4. El manejo explícito de excepciones convierte errores potencialmente silenciosos en fallos controlados y trazables.
5. La combinación de los tres pilares produce un sistema significativamente más robusto que una implementación estructurada equivalente.

Referencias

Python Software Foundation. (2024). Python documentation. <https://docs.python.org>

Lutz, M. (2021). Learning Python (5th ed.). O'Reilly Media.

Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.